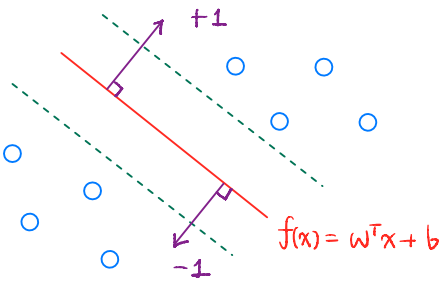
Support Vector Machine (SVM)

단순 logistic regression 보다는 margin을 최대화 하는 decision boundary 찾는 것 — 제약사항

- find a decision boundary with the maximum margin
  - margin: distance from the boundary to the closest training points
- supervised learning model for classification (and regression as SVR)
- linear decision function
  - $f(x) = w^\top x + b$
  - predict by $\text{sign}(f(x))$



Maximum-margin (hard margin)

- separable case
  - constraints: $y_i(w^\top x_i + b) \geq 1$
  - maximize margin $\frac{2}{|w|}$ is equivalent to minimizing $|w|^2$
- primal optimization
  - $\min_{w,b}; \frac{1}{2}|w|^2$
  - subject to $y_i(w^\top x_i + b) \geq 1$
- support vectors
  - the points that lie on the margin boundaries
  - they "support" the optimal hyperplane
  - only these points determine the solution

평면 $w^Tx + b = c_1$ 사이의 거리 공식 → margin = $\frac{|c_1 - c_2|}{\|w\|}$

$w^Tx + b = c_2$ $c_1 = 1$ $c_2 = -1$

$\|w\|^2 \downarrow \rightarrow$ margin $\uparrow$

hard margin: 모든 data가 margin 밖에 있도록

$y \in \{-1, +1\}$

SVM은 "모든 sample이 margin 밖에 있어야 한다"는 제약이 있는 optimization 문제

$y_i(w^Tx_i + b) \geq 1$

Soft margin (non-separable)

현실적으로 hard margin 불가능할 때가 많음 → 약간의 위반량 허용

- allow violations via slack variables ξ_i
  - constraints: $y_i(w^\top x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$
- objective
  - $\min_{w,b}; \frac{1}{2}|w|^2 + C\sum_i \xi_i$

C ↑ → 위반량의 가중치 ↑ → 더 강하게 penalty

C ↓ → 위반량의 가중치 ↓ → 위반량 허용 ↑ w/ large margin

    - C controls tradeoff
        - $C \uparrow \rightarrow$ fewer violations, tighter fit (variance ↑)
        - $C \downarrow \rightarrow$ larger margin, more tolerance (bias ↑)
- hinge loss view
    - equivalent form:
        - $\min_{w,b}; \frac{1}{2}|w|^2 + C\sum_i \max(0, 1 - y_i f(x_i))$
    - margin violations are penalized linearly

$y_i f(x_i) \geq 1 \rightarrow$ margin 규칙 잘 만족

margin 위반할 정도만큼 penalty 줌

Kernel trick

- handle nonlinear boundaries by mapping inputs to a higher-dimensional feature space
    - explicit mapping $\phi(x)$ is not computed
    - use kernel $K(x, x') = \phi(x)^\top \phi(x')$
- dual form depends only on dot products
    - prediction:
        - $f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$

kernel 사용 → 데이터 간 내적으로 표현

각 data sample이 support vector와 얼마나 유사한지

- common kernels
    - linear: $K(x, x') = x^\top x'$
    - RBF: $K(x, x') = \exp(-\gamma|x - x'|^2)$
    - polynomial: $K(x, x') = (x^\top x' + c)^d$

primal: 직접 변수 w, b을 최적화

경계의 parameter 직접 찾기

* dual: 각 data point에 대응하는 변수 α_i 최적화

각 sample이 경계를 얼마나 밀고 당기는지

→ w: data의 linear combination

* duality: optimization 문제를 다른 변수로 다시 씀

Lagrangian minimize $f(x)$ + subject to $h(x) = 0$ or $g(x) \leq 0$

$\mathcal{L}(x, \lambda) = f(x) + \lambda h(x)$ λ: Lagrangian multiplier

$\mathcal{L}(x, \alpha) = f(x) + \alpha g(x)$ where $\alpha \geq 0$

SVM에서는 Lagrangian 관점에서

$g(x) = 1 - y_i f(x_i) \leq 0$

$\min_x \max_{\alpha \geq 0} \mathcal{L}(x, \alpha)$ dual problem 1) α 고정하고 $\mathcal{L}$을 최소화하는 x 찾기

2) 그 x를 고정해놓고 $\mathcal{L}$을 최대화하는 α 찾기

Initialization

- choosing initial values of model parameters before training
    - affect gradient flow & training stability

    - e.g., ReLU, Leaky ReLU, GELU
- compensate for ReLU's sparsity
    - about half of activations are zero
    - scaling by 2 → help preserve activation variance & gradient flow

---

Logit & Softmax

Logit

$\text{logit}(p) = \log \frac{p}{1-p}$

- logarithm of the odds of the event
    - raw & unnormalized scores output by the model
    - pre-softmax outputs whose differences correspond to log-odds between classes
- interpreted as evidence for each class

Softmax

$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$

- convert logits into a probability distribution
    - output is positive and sums to 1
    - preserve relative differences between logits
- smooth and differentiable
    - compatible with likelihood-based objectives
- e^z grows extremely fast
    - if some logit z is large, e^z can exceed floating-point range → overflow (`inf`)

Log-sum-exp trick

exp 각 항에서 최댓값 빼 줌 → overflow 방지

$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}, m = \max_j z_j$ 최댓값 빼줌

- subtract the maximum logit to avoid overflow
  - keep exponentials in a safe range
  - $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$
  - $z' = z - \max_j z_j$
  - $\text{softmax}(z) = \text{softmax}(z')$

$$\frac{e^{z_i - c}}{\sum_j e^{z_j - c}}$$

Gumbel Softmax

$y_i = \frac{\exp((\log p_i + g_i)/\tau)}{\sum_j \exp((\log p_j + g_j)/\tau)}$

- differentiable approximation to categorical sampling
  - sampling from a categorical distribution is non-differentiable
  - differentiable with respect to logits
- use cases
  - discrete latent variables
  - VQ-VAE style model

Entropy

$H(p) = -\sum_i p_i \log p_i$

- measure uncertainty of a distribution
- defined as the expected amount of surprise
  - surprisal of an event: $-\log p(x)$
- entropy ↑ → uncertainty ↑ → ≈ uniform distibution

- entropy ↓ → more confident or peaked distribution → ≈deterministic distribution

Cross-Entropy

$H(p, q) = -\sum_i p_i \log q_i$

- measures the expected surprisal when
  - true distribution is p
  - but outcomes are encoded using model distribution q
  - how "surprised" we are when using q to explain data from p
- equivalent to negative log-likelihood
  - used as the standard loss for classification
  - penalizes assigning low probability to true labels

KL Divergence

$\mathrm{KL}(p||q) = \sum_i p_i \log \frac{p_i}{q_i} = H(p, q) - H(p)$

- measure how much information is lost when q is used to approximate p
  - extra surprise due to mismatch between p and q
- minimizing cross-entropy ⇔ minimizing KL divergence
  - since $H(p)$ is fixed
    - entropy $H(p)$: intrinsic uncertainty of the data

Dropout

- randomly deactivate neurons during training
  - prevent the network from relying too heavily on specific activations → overfitting ↓